

Plano do workshop

Attention is All You Need: construindo um Transformer do zero

Giovanni Vasconcelos · CESUPA Tech Summit 2026

Sumário

1. Visão geral	1
2. Preparação	2
D – 14: repositório	2
D – 10: modelo de referência	3
D – 7: ensaio	3
D – 3: logística	3
Dia D	3
3. Roteiro minuto a minuto	3
4. Condução da turma	4
5. Erros comuns (para os monitores)	5
6. Riscos e contingências	5
7. Estrutura do repositório	6

1. Visão geral

Objetivo. Ao final das duas horas, cada participante terá implementado os componentes centrais de um Transformer decoder-only (atenção, multi-head attention e o bloco com conexões residuais) e usado esse código para treinar um modelo de linguagem que gera partidas de xadrez. O foco é a compreensão do mecanismo, não o desempenho do modelo.

Público. Estudantes de todos os semestres. O roteiro foi desenhado para quem já teve contato com programação em Python e noções de álgebra linear. Estudantes dos primeiros semestres acompanham a intuição pelos slides e usam as células de recuperação do notebook para não perder o fio. Espera-se uma turma de mais de 40 pessoas, com pelo menos dois monitores.

Formato. Metade exposição, metade prática, sempre alternadas: cada bloco teórico precede imediatamente o exercício correspondente. Toda a parte prática acontece no Google Colab. O participante

precisa apenas de um navegador e de uma conta Google; nada é instalado nos computadores do laboratório.

Materiais.

Arquivo	Função
workshop_aluno.ipynb	Notebook da turma, com seis exercícios e células de verificação
workshop_gabarito.ipynb	Mesmo notebook com as soluções; para o ministrante e os monitores
pretreino_referencia.ipynb	Treina o modelo de referência no Colab (executar uma vez, antes do evento)
mini_gpt.py	Implementação de referência, importada pelas células de recuperação
checagens.py	Verificações automáticas dos exercícios
slides/attention_workshop.pptx	Apresentação (30 slides, com notas do apresentador em todos)
docs/GUIA_DE_ESTUDOS.md	Guia de estudos para a preparação do ministrante
data/xadrez.txt.gz	60 mil partidas do Lichess (2013), já filtradas e validadas

2. Preparação

As datas abaixo são relativas ao dia do workshop (D).

D – 14: repositório

1. O repositório público já existe: <https://github.com/giovannibragasv/transformer-do-zero>. O usuário giovannibragasv já está configurado em `scripts/gerar_notebooks.py`, `slides/gerar_slides.js` e `README.md`.
2. Se algo mudar, regenere os artefatos:

```
.venv/bin/python scripts/gerar_notebooks.py  
cd slides && npm install && node gerar_slides.js attention_workshop.pptx
```
3. Fazer o commit de tudo, exceto `data_raw/`, `data/xadrez.txt` e `.venv/` (já listados no `.gitignore`).
4. Abrir https://colab.research.google.com/github/giovannibragasv/transformer-do-zero/blob/main/workshop_aluno.ipynb e confirmar que o notebook abre e que a primeira célula clona o repositório.
5. Gerar um link curto (e, se possível, um QR code) para esse endereço e atualizar o slide 4.

D – 10: modelo de referência

1. Abrir `pretreino_referencia.ipynb` no Colab com GPU.
2. Executar as células. O treino dura 60 minutos e salva checkpoints a cada 10 minutos.
3. Baixar `xadrez_referencia.pt` (cerca de 19 MB), colocar em `modelos/` e fazer o commit.
4. Anotar a taxa de legalidade impressa ao final. Ela entra na fala do slide 25 e serve de referência para a seção 3.4 do notebook.

Se o treino for interrompido, o último checkpoint salvo já é utilizável. Se o modelo não estiver disponível no dia, o notebook usa automaticamente o modelo treinado pela turma.

D – 7: ensaio

1. Com uma conta Google diferente da sua, sem nada salvo no Drive, executar o `workshop_aluno.ipynb` do início ao fim como se fosse um participante. Resolver os exercícios sem consultar o gabarito e cronometrar cada parte.
2. Executar o `workshop_gabarito.ipynb` inteiro e salvar a versão executada (com as saídas). Ela serve para demonstração caso o Colab fique instável no dia.
3. Ensaiar a exposição com os slides em voz alta. As notas do apresentador trazem o conteúdo de cada fala; o tempo de cada bloco teórico é de 10 minutos.
4. Fazer o estudo descrito no guia (`GUIA_DE_ESTUDOS.md`), em especial a seção de perguntas difíceis.

D – 3: logística

- Confirmar com a organização o laboratório, o número de máquinas, o projetor e a conexão com a internet. Cada participante baixa cerca de 30 MB na primeira célula; para 40 pessoas, são pouco mais de 1 GB em alguns minutos.
- Confirmar que o navegador dos laboratórios permite login em contas Google.
- Reunir os monitores por 30 minutos: percorrer o gabarito, combinar a divisão da sala e apresentar a lista de erros comuns (seção 5).
- Preparar um cartaz ou slide com o link curto do notebook.

Dia D

- Chegar 30 minutos antes. Testar o projetor com os slides e com o Colab (fonte do notebook ampliada para 125% ou mais).
- Deixar abertos em abas: slides, notebook do aluno, gabarito executado e o notebook de gabarito pronto para a demonstração ao vivo do slide 3.
- Conectar-se a uma GPU no Colab antes do início, para a demonstração.

3. Roteiro minuto a minuto

Horário	Slides	Notebook	Atividade
00:00	1–3		Apresentação, combinados e demonstração: jogar três lances contra o modelo de referência
00:05	4	Seção 0	Acesso ao Colab: abrir o link, salvar cópia, ativar a GPU, executar a seção 0. Aguardar a turma inteira
00:15	5–12		Motivação (RNNs, Tabela 1 do artigo) e atenção: dicionário, Q/K/V, a equação, $\sqrt{d_k}$, máscara causal
00:25	13	Parte 1	Exercícios 1 a 3 em NumPy. Aviso aos 00:40
00:45	14–19		Multi-head, posição, bloco, arquitetura original, famílias e o modelo completo
00:55	20	Parte 2	Exercícios 4 a 6 em PyTorch e treino da ordenação
01:10	21	2.6	Discutir o mapa de atenção da ordenação com a turma
01:13	22	3.1, 3.2	Todos iniciam o treino do modelo de xadrez
01:15	22		Intervalo de 5 minutos (o treino roda durante a pausa)
01:20	23–26		Xadrez como linguagem, objetivo de treino, modelos de mundo, temperatura
01:30	27	3.3–3.6	Gerar partidas, variar a temperatura, comparar com o modelo de referência, jogar
01:50	28–30		Daqui até um LLM, referências e perguntas
02:00			Encerramento

Pontos de controle. Em três momentos o ministrante deve verificar o estado da sala antes de avançar: ao fim da seção 0 (todos com GPU), aos 00:45 (todos com a Parte 1 concluída ou recuperada) e aos 01:13 (todos com o treino de xadrez em execução). Nesses momentos, peça que quem ainda não chegou ao ponto execute as células de recuperação.

Margem. O roteiro não tem folga explícita. Se houver atraso, reduza primeiro o slide 18 (famílias) e o slide 15 (posição), que podem ser tratados em um minuto cada, e depois a seção 3.6 do notebook, que é complementar.

4. Condução da turma

Divisão da sala. Com 40 ou mais participantes, divida o laboratório em zonas, uma por monitor. O ministrante fica com a frente da sala e circula apenas durante a Parte 2, que é a mais difícil.

Sinal de ajuda. Combine um sinal visível e silencioso, por exemplo um papel dobrado sobre o monitor. Isso evita que o ministrante seja interrompido durante a exposição e permite que os monitores priorizem.

Heterogeneidade. Para estudantes dos primeiros semestres, a orientação é explícita: o objetivo é entender o que cada exercício faz, não necessariamente escrevê-lo sozinho. Após três minutos

parados em um exercício, eles devem usar a célula de recuperação e seguir. Incentive duplas mistas (um estudante mais avançado e um iniciante), que funcionam bem nesse formato.

Estudantes adiantados. Quem terminar antes pode fazer os exercícios para casa do final do notebook já durante o workshop: trocar o embedding de posição pelo senoidal ou aumentar o modelo.

5. Erros comuns (para os monitores)

Exercício	Sintoma	Causa provável
1	“a soma deve ser feita ao longo do último eixo”	Falta <code>keepdims=True</code> ou <code>axis=-1</code>
1	NaN com entradas grandes	Não subtraiu o máximo antes de <code>np.exp</code>
2	Erro de shape em <code>Q @ K</code>	Falta <code>transpor K</code>
2	“não foram divididos por $\sqrt{d_k}$ ”	Esqueceu a escala
3	Pesos errados com máscara	Máscara com <code>np.triu</code> em vez de <code>np.tril</code>
4	Shape <code>(T, T, B)</code> ou similar	Usou <code>k.T</code> , que inverte todas as dimensões em tensores 3D
4	Erro de tamanho na máscara	Usou <code>self.tril</code> sem recortar <code>[:T, :T]</code>
5	Shape <code>(B, T, head_size)</code>	Concatenou no eixo errado; deve ser <code>dim=-1</code>
6	“o bloco devolveu a entrada sem alterações”	As duas linhas não foram escritas
6	“faltam as conexões residuais”	Escreveu <code>x = self.attn(...)</code> em vez de <code>x = x + self.attn(...)</code>
Geral	<code>NameError</code> em células posteriores	A pessoa pulou uma célula; executar tudo até o ponto atual (<i>Ambiente de execução > Executar anteriores</i>)

6. Riscos e contingências

Risco	Contingência
Colab sem GPU disponível para parte da turma	Partes 1 e 2 funcionam em CPU. Na Parte 3, a pessoa interrompe o treino e usa o modelo de referência (seção 3.4), ou trabalha em dupla
Internet lenta no laboratório	Distribuir o repositório em pen drive como plano B; o notebook funciona após <i>Arquivos > Upload</i> do zip e descompactação
Colab fora do ar	Projetar o gabarito já executado (salvo no ensaio) e conduzir a Parte 3 como demonstração
Modelo de referência ausente	O notebook usa automaticamente o modelo treinado pela turma; a comparação da seção 3.4 fica prejudicada, mas nada quebra

Risco	Contingência
Atraso acumulado	Cortar conforme a seção 3 (slides 18 e 15, depois a seção 3.6)
Turma muito heterogênea	Reforçar o uso das células de recuperação; os monitores priorizam quem está parado na Parte 2

7. Estrutura do repositório

```

transformer-do-zero/
├─ workshop_aluno.ipynb      notebook da turma (gerado)
├─ workshop_gabarito.ipynb   soluções (gerado)
├─ pretreino_referencia.ipynb treino do modelo de referência (gerado)
├─ mini_gpt.py               implementação de referência
├─ checagens.py              verificações dos exercícios
├─ data/xadrez.txt.gz         conjunto de dados
├─ modelos/                  xadrez_referencia.pt (após o pré-treino)
├─ scripts/
│   ├─ preparar_dados.py      PGN do Lichess -> data/xadrez.txt
│   ├─ pretreinar.py          treino do modelo de referência
│   └─ gerar_notebooks.py     fonte única dos notebooks
├─ slides/
│   ├─ attention_workshop.pptx
│   ├─ gerar_slides.js        fonte da apresentação (pptxgenjs)
│   └─ assets/                equações e gráficos
└─ docs/                     plano e guia de estudos (.md e .tex)

```

Os notebooks e os slides são gerados a partir de scripts. Qualquer alteração deve ser feita na fonte (scripts/gerar_notebooks.py ou slides/gerar_slides.js) e o artefato regenerado. Editar o .ipynb diretamente funciona, mas a alteração se perde na próxima geração.

Testes. O comando abaixo executa o gabarito inteiro em modo rápido (treinos de poucos passos, em CPU) e falha se alguma célula levantar erro:

```

WORKSHOP_RAPIDO=1 .venv/bin/jupyter nbconvert --to notebook --execute \
    workshop_gabarito.ipynb --output /tmp/teste.ipynb

```